

National University of Technology

Artificial Neural Network Lab- CS3512

LAB Submitted to:

Maj. Rooshan Saleem

Submitted By:

Muhammad Abdullah

Reg. No: **F22607010**
Batch: **Fall AI 2022**
Lab : **01**

Q.NO.1:

Apply a Logistic Regression and Neural Network on Mnist dataset.

Answer:

Importing the dataset and Printing data labels on Train and test.

```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) =
mnist.load_data()

print(f"Training data shape: {train_images.shape}, Training labels shape:
{train_labels.shape}")
print(f"Test data shape: {test_images.shape}, Test labels shape:
{test_labels.shape}")
```

```
Training data shape: (60000, 28, 28), Training labels shape: (60000,)
Test data shape: (10000, 28, 28), Test labels shape: (10000,)
```

Applying a Linear Regression and checking Accuracy Score.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

train_images_flat = train_images.reshape((60000, 28 * 28))
test_images_flat = test_images.reshape((10000, 28 * 28))

logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(train_images_flat, train_labels.argmax(axis=1))

test_predictions = logistic_model.predict(test_images_flat)

logistic_test_acc = accuracy_score(test_labels.argmax(axis=1),
test_predictions)
print(f'Logistic Regression Test accuracy: {logistic_test_acc}')
```

```
Logistic Regression Test accuracy: 0.9256
```

Applying a Neural Network and checking accuracy on 10 Epochs.

```
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.utils import to_categorical

train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32')
/ 255
test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') /
255

train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)

model = Sequential([
    Flatten(input_shape=(28, 28, 1)),
    Dense(512, activation='relu'),
    Dense(10, activation='softmax')
])


model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'])


model.fit(train_images, train_labels, epochs=10, batch_size=128)

test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f'Test accuracy: {test_acc}')
```

```
469/469 [==============================] - 5s 8ms/step - loss: 0.2624 - accuracy: 0.9235
Epoch 2/10
469/469 [==============================] - 5s 10ms/step - loss: 0.1062 - accuracy: 0.9688
Epoch 3/10
469/469 [==============================] - 5s 10ms/step - loss: 0.0707 - accuracy: 0.9792
Epoch 4/10
469/469 [==============================] - 3s 7ms/step - loss: 0.0503 - accuracy: 0.9851
Epoch 5/10
469/469 [==============================] - 3s 7ms/step - loss: 0.0382 - accuracy: 0.9886
Epoch 6/10
469/469 [==============================] - 4s 8ms/step - loss: 0.0295 - accuracy: 0.9911
Epoch 7/10
469/469 [==============================] - 4s 8ms/step - loss: 0.0223 - accuracy: 0.9936
Epoch 8/10
469/469 [==============================] - 3s 6ms/step - loss: 0.0163 - accuracy: 0.9955
Epoch 9/10
...
Epoch 10/10
469/469 [==============================] - 3s 7ms/step - loss: 0.0098 - accuracy: 0.9973
313/313 [==============================] - 1s 2ms/step - loss: 0.0742 - accuracy: 0.9787
Test accuracy: 0.9786999821662903
```